Estimation_par_noyeau

Amadou Bocoum - Anya Levêque

2025-12-14

Implementation de la fonction f et de sa derivée

$f(x) = (x+2)\sin(4\pi x^2) + 2\sin(8\pi x)$ et

$f'(x) = \sin(4\pi x^2) + 8\pi x(x+2)\cos(4\pi x^2) + 16\pi\cos(8\pi x)$

```
f=function(x){ ( (x+2)*sin(4*pi*x^2)+2*sin(8*pi*x)  )}

fprime = function(x){
sin(4*pi*x^2)+(8*pi*x)*(x + 2)*cos(4*pi*x^2 )+16*pi*cos(8*pi*x)
}
```

Implementation des trois noyeaux et leurs dérivées:

- Noyau d'**Epanechnikov** :

```
Epach=function(x){ ((3/4)*(1-x^2))*(abs(x)<=1) }

#Sa derivée:

kprime_Epach=function(x){ (-3/2*x) * (abs(x)<=1) }
```

- Noyau **quadratique** :

```
quadratique=function(x){ ((15/16)*(1-x^2)^2)*(abs(x)<=1) }

#Sa derivée:

kprime_quadratique=function(x){ (-15/4 * x * (1-x^2)) * (abs(x)<=1) }
```

- Noyau **gaussien** :

```
gaussian=function(x){dnorm(x)}

#Sa derivée

kprime_gaussian = function(x){-x*dnorm(x)}
```

Implementation des estimateurs ($\hat{h}_n(x)$ et $\hat{g}_n(x)$) et leurs derivées:

```
#hchapeau:

h_hat=function(d,X,Y,alpha,k){
  n=length(X)
  h=(1:n)^(-alpha)
  mean(Y * k((d-X)/h) / h)
}


#Sa derivée

h_hatprime=function(d,X,Y,alpha,kprime){
  n=length(X)
  h=(1:n)^(-alpha)
  mean(Y*kprime((d-X)/h)/h^2)
}

# g_chapeau:
g_hat=function(d,X,alpha,k){
  n=length(X)
  h=(1:n)^(-alpha)
  mean(k((d-X)/h)/h)
}

# Sa derivée:
g_hatprime=function(d,X,alpha,kprime){
  n=length(X)
  h=(1:n)^(-alpha)
  mean(kprime((d-X)/h)/h^2)
}
```

Reponse à la question 1)

Implementation des trois estimateurs de $f^{'}(x)$:

Equation (9):

```
f_hatprime=function(d,X,Y,alpha,k,kprime){
num1= h_hatprime(d,X,Y,alpha,kprime)
den1 = g_hat(d,X,alpha,k)

num2 = h_hat(d,X,Y,alpha,k)*g_hatprime(d,X,alpha,kprime)
den2=g_hat(d,X,alpha,k)^2
resultat= (num1/den1) - (num2/den2)
return(resultat)

}
```

Equation (10):

```
f_tildeprime = function(d,X, Y,alpha,g,gprime,k,kprime){
num1= h_hatprime(d,X,Y,alpha,kprime)
den1 = g(d)
num2 = h_hat(d,X,Y,alpha,k)*gprime(d)
den2=(g(d))^2
resultat= (num1/den1) - (num2/den2)
return(resultat)
}
```

Equation (11)

```
f_qprime = function(d, X, Y, alpha, g, kprime){
  n <- length(X)
  h <- (1:n)^(-alpha)
  mean( Y * kprime((d - X)/h) / (g(X) * h^2) )
}
```

Reponse à la question 2)

Question 2a)

Simulation des données:

```
set.seed(123)

Nmax= 10000
X_full =runif(Nmax)
Y_full = f(X_full)+rnorm(Nmax)

# 1) Paramètres
n <- 10000
alpha <- 0.3 # valeur prise en fonction de l'article jusque là (avant la cross validation de la dernièr
X <- X_full[1:n]
Y <- Y_full[1:n]

# 2) Grille
x_grid <- seq(0, 1, length.out = 200)

# 3) Vraie dérivée
fprime_true_values <- fprime(x_grid)

# 4) Estimateurs
fprime_hat_values <- sapply(x_grid, function(x0){
  f_hatprime(
    d = x0,
    X = X,
```

```
    Y = Y,
    alpha = alpha,
    k = Epach,
    kprime = kprime_Epach
  )
})

fprime_tilde_values <- sapply(x_grid, function(x0){
  f_tildeprime(
    d = x0,
    X = X,
    Y = Y,
    alpha = alpha,
    g = function(z) 1,
    gprime = function(z) 0,
    k = Epach,
    kprime = kprime_Epach
  )
})

fprime_q_values <- sapply(x_grid, function(x0){
  f_qprime(
    d = x0,
    X = X,
    Y = Y,
    alpha = alpha,
    g = function(z) 1,
    kprime = kprime_Epach
  )
})
```

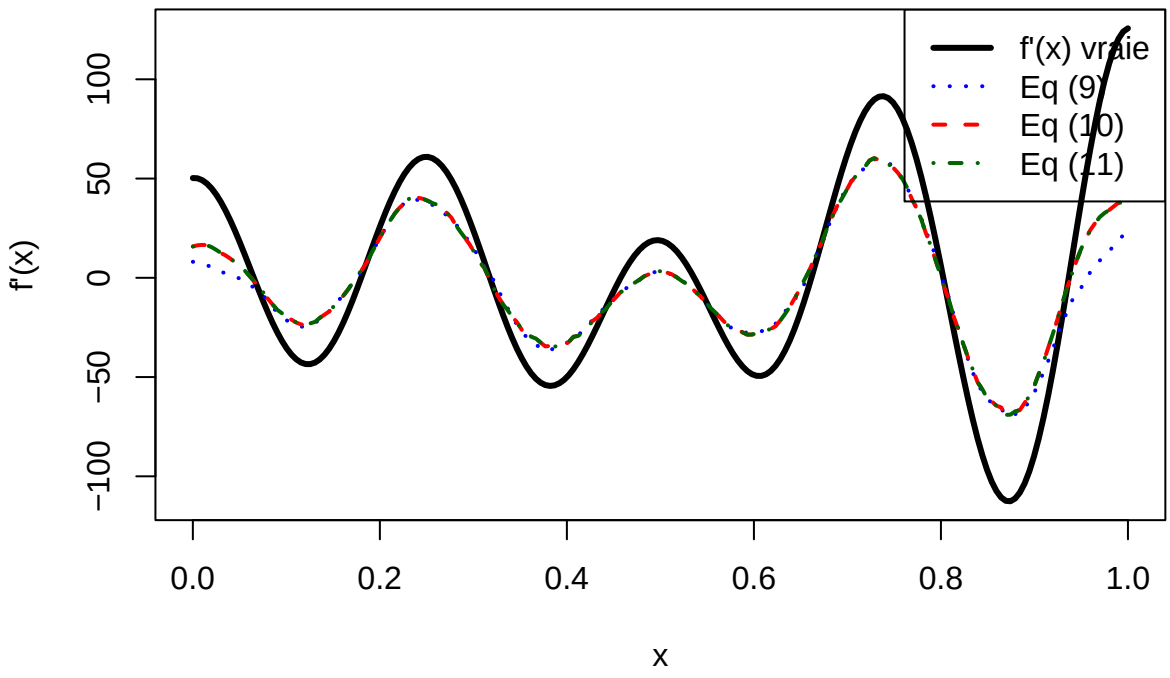
Graphique:

```
plot(x_grid, fprime_true_values, type="l", lwd=3, col="black",
     xlab="x", ylab="f'(x)",
     main="Figure 1 : convergence presque sûre")

lines(x_grid, fprime_hat_values, col="blue", lwd=2, lty=3)
lines(x_grid, fprime_tilde_values, col="red", lwd=2, lty=2)
lines(x_grid, fprime_q_values, col="darkgreen", lwd=2, lty=4)

legend("topright",
       legend=c("f'(x) vraie", "Eq (9)", "Eq (10)", "Eq (11)"),
       col=c("black","blue","red","darkgreen"),
       lwd=c(3,2,2,2),
       lty=c(1,3,2,4))
```

Figure 1 : convergence presque sûre



Question 2b: TCL:

Calcul de la constante théorique :

```
xi2 <- integrate(function(u) kprime_Epach(u)^2, -1, 1)$value
```

Parametres de simulation :

```
n=500
alpha=0.3
hn = n^(-alpha)

N=2000
x_vals =c(0.5,0.8)
```

```
# Simulation d'une seule réalisation:

one=function(x0, estimator, ...){
  X <- runif(n)
  Y <- f(X) + rnorm(n)
  est <- estimator(d = x0, X = X, Y = Y, alpha = alpha, ...)
  sqrt(n * hn^3) * (est - fprime(x0))
}
```

```r
# Repetition pour chacune des equations 9,10 et 11 avec les deux valeurs données (0.5 et 0.8)

Z9_05  <- replicate(N, one(0.5, f_hatprime,
                           k = Epach, kprime = kprime_Epach))
Z9_08  <- replicate(N, one(0.8, f_hatprime,
                           k = Epach, kprime = kprime_Epach))

Z10_05 <- replicate(N, one(0.5, f_tildeprime,
                           g = function(z) 1,
                           gprime = function(z) 0,
                           k = Epach, kprime = kprime_Epach))
Z10_08 <- replicate(N, one(0.8, f_tildeprime,
                           g = function(z) 1,
                           gprime = function(z) 0,
                           k = Epach, kprime = kprime_Epach))

Z11_05 <- replicate(N, one(0.5, f_qprime,
                           g = function(z) 1,
                           kprime = kprime_Epach))
Z11_08 <- replicate(N, one(0.8, f_qprime,
                           g = function(z) 1,
                           kprime = kprime_Epach))
```

Les variances Théoriques :

```r
var9  <- function(x) xi2/(1 + 3*alpha)
var10 <- function(x) xi2/(1 + 3*alpha) * (f(x)^2 + 1)
var11 <- function(x) xi2/(1 + 3*alpha) * (f(x)^2 + 1)
```

Figure 2 de la question 2.b

Remarque importante :

conformement au théorème 3.2,la variable

$$Z_n(x) = \sqrt{nh_n^3}\left(\widehat{f}'(x) - f'(x)\right).$$

converge en loi vers une loi normale centrée. En pratique pour des tailles d'échantillons finies, la convergence est lente et un biais peut encore subsister. Afin de mieux illustrer la normalité asymptotique, les realisations sont recentrées empiriquement sans modifier la variance théorique

```r
par(mfrow = c(3,2))

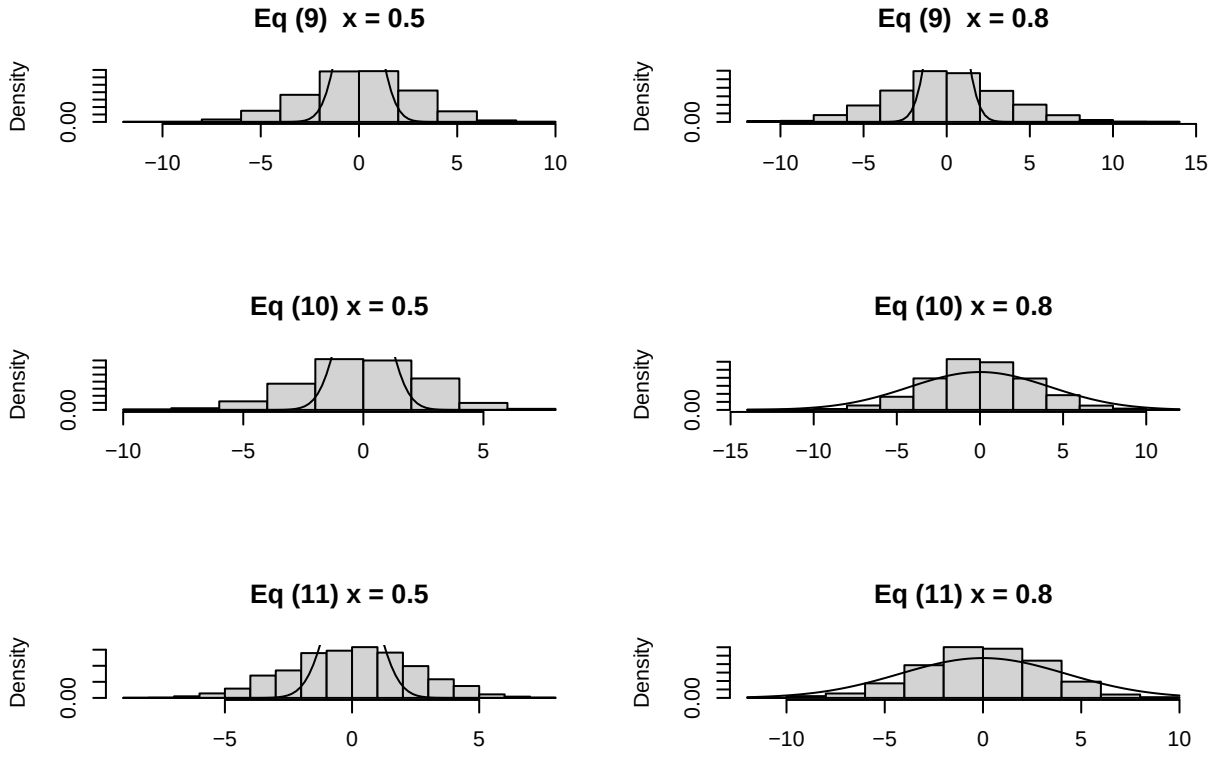
plot_hist <- function(Z, sd_th, main){
  Zs <- Z - mean(Z)
  hist(Zs, breaks = 12, freq = FALSE, main = main, xlab = "")

  #hist(Z, breaks = 12, freq = FALSE, main = main, xlab = "")
  curve(dnorm(x, 0, sd_th), add = TRUE)
}
```

```r
plot_hist(Z9_05,  sqrt(var9(0.5)),  "Eq (9)  x = 0.5")
plot_hist(Z9_08,  sqrt(var9(0.8)),  "Eq (9)  x = 0.8")

plot_hist(Z10_05, sqrt(var10(0.5)), "Eq (10) x = 0.5")
plot_hist(Z10_08, sqrt(var10(0.8)), "Eq (10) x = 0.8")

plot_hist(Z11_05, sqrt(var11(0.5)), "Eq (11) x = 0.5")
plot_hist(Z11_08, sqrt(var11(0.8)), "Eq (11) x = 0.8")
```




```r
par(mfrow = c(1,1))
```

Question 3: La MSE et la Cross-Validation:

Cross - Validation:

```r
CV_alpha <- function(alpha, X, Y, K, Kprime, estimator = c("eq9","eq10","eq11")) {

  estimator <- match.arg(estimator)
  n <- length(X)
  cv_sum <- 0

  for (k in 1:n) {
```

```
    X_m <- X[-k]
    Y_m <- Y[-k]
    xk  <- X[k]

    if (estimator == "eq9") {
      f_hat <- f_hatprime(
        d = xk, X = X_m, Y = Y_m,
        alpha = alpha, k = K, kprime = Kprime
      )
    }

    if (estimator == "eq10") {
      f_hat <- f_tildeprime(
        d = xk, X = X_m, Y = Y_m,
        alpha = alpha,
        g = function(z) 1,
        gprime = function(z) 0,
        k = K, kprime = Kprime
      )
    }

    if (estimator == "eq11") {
      f_hat <- f_qprime(
        d = xk, X = X_m, Y = Y_m,
        alpha = alpha,
        g = function(z) 1,
        kprime = Kprime
      )
    }

    cv_sum <- cv_sum + (f_hat - fprime(xk))^2
  }

  cv_sum / n
}
```

Calcul du α optimal en fonction de l'estimateur et du noyeau (Le noyeau d'Epanechnikov seul utilisé ici)

```
# le meilleur alpha pour l'estimateur 9

alpha_grid <- seq(0.20, 0.30, by = 0.02)

CV_eq9_epa <- sapply(alpha_grid, function(a)
  CV_alpha(a, X, Y, Epach, kprime_Epach, estimator = "eq9")
)

alpha_opt <- alpha_grid[which.min(CV_eq9_epa)]
alpha_opt
```

```
## [1] 0.3
```

```r
# le meilleur alpha pour l'estimateur 10

alpha_grid <- seq(0.20, 0.30, by = 0.02)

CV_eq10_epa <- sapply(alpha_grid, function(a)
  CV_alpha(a, X, Y, Epach, kprime_Epach, estimator = "eq10")
)

alpha_opt <- alpha_grid[which.min(CV_eq10_epa)]
alpha_opt
```

```
## [1] 0.3
```

```r
# le meilleur alpha pour l'estimateur 11

alpha_grid <- seq(0.20, 0.30, by = 0.02)

CV_eq11_epa <- sapply(alpha_grid, function(a)
  CV_alpha(a, X, Y, Epach, kprime_Epach, estimator = "eq11")
)

alpha_opt <- alpha_grid[which.min(CV_eq11_epa)]
alpha_opt
```

```
## [1] 0.3
```

La validation croisée (alpha_opt obtenus pour les equations 9,10 et 11 avec le noyeau d'Epanechnikov) donne la même valeur optimale de alpha pour les trois estimateurs. Nous avons également testé les noyaux quadratique et gaussien et obtenu la même valeur.C'est pourquoi nous fixons alpha pour la suite des algorithmes

```r
set.seed(123)

# Paramètres globaux
n <- 10000
M <- 200
alpha <- alpha_opt   # alpha Fixé après Cross-Validation
x_points <- c(0.5, 0.8)

# Fonctions g et g'

g_unif  <- function(x) 1
gp_unif <- function(x) 0

# Noyaux

kernels <- list(
  Epanechnikov = list(K = Epach,       Kp = kprime_Epach),
  Quadratique  = list(K = quadratique, Kp = kprime_quadratique),
  Gaussien     = list(K = gaussian,    Kp = kprime_gaussian)
)
```

```r
# Estimateurs

est_eq9 <- function(x0, X, Y, alpha, K, Kp) {
  f_hatprime(d = x0, X = X, Y = Y,
             alpha = alpha, k = K, kprime = Kp)
}

est_eq10 <- function(x0, X, Y, alpha, K, Kp) {
  f_tildeprime(d = x0, X = X, Y = Y,
               alpha = alpha,
               g = g_unif, gprime = gp_unif,
               k = K, kprime = Kp)
}

est_eq11 <- function(x0, X, Y, alpha, Kp) {
  f_qprime(d = x0, X = X, Y = Y,
           alpha = alpha,
           g = g_unif, kprime = Kp)
}

# Stockage des résultats

MSE_results <- list()

# Boucle principale de la MSE

for (ker in names(kernels)) {

  K  <- kernels[[ker]]$K
  Kp <- kernels[[ker]]$Kp

  mse_matrix <- matrix(NA, nrow = length(x_points), ncol = 3)
  colnames(mse_matrix) <- c("Eq (9)", "Eq (10)", "Eq (11)")
  rownames(mse_matrix) <- paste0("x = ", x_points)

  for (j in seq_along(x_points)) {

    x0 <- x_points[j]
    true_val <- fprime(x0)

    err9  <- numeric(M)
    err10 <- numeric(M)
    err11 <- numeric(M)

    for (m in 1:M) {

      # Simulation des donnees
      X <- runif(n)
      Y <- f(X) + rnorm(n)

      # Estimateurs
      est9  <- est_eq9(x0, X, Y, alpha, K, Kp)
      est10 <- est_eq10(x0, X, Y, alpha, K, Kp)
```

```
      est11 <- est_eq11(x0, X, Y, alpha, Kp)

      # Erreurs quadratiques
      err9[m]  <- (est9  - true_val)^2
      err10[m] <- (est10 - true_val)^2
      err11[m] <- (est11 - true_val)^2
    }

    mse_matrix[j, "Eq (9)"]  <- mean(err9)
    mse_matrix[j, "Eq (10)"] <- mean(err10)
    mse_matrix[j, "Eq (11)"] <- mean(err11)
  }

  MSE_results[[ker]] <- as.data.frame(mse_matrix)
}

# Résultats finaux
MSE_results
```

```
## $Epanechnikov
##            Eq (9)   Eq (10)   Eq (11)
## x = 0.5 267.37226 267.51550 267.51550
## x = 0.8  36.66619  35.32172  35.32172
##
## $Quadratique
##            Eq (9)   Eq (10)   Eq (11)
## x = 0.5 160.61909 160.57077 160.57077
## x = 0.8  22.34596  25.94399  25.94399
##
## $Gaussien
##            Eq (9)   Eq (10)   Eq (11)
## x = 0.5 842.68270 841.80178 841.80178
## x = 0.8  61.42862  66.02729  66.02729
```